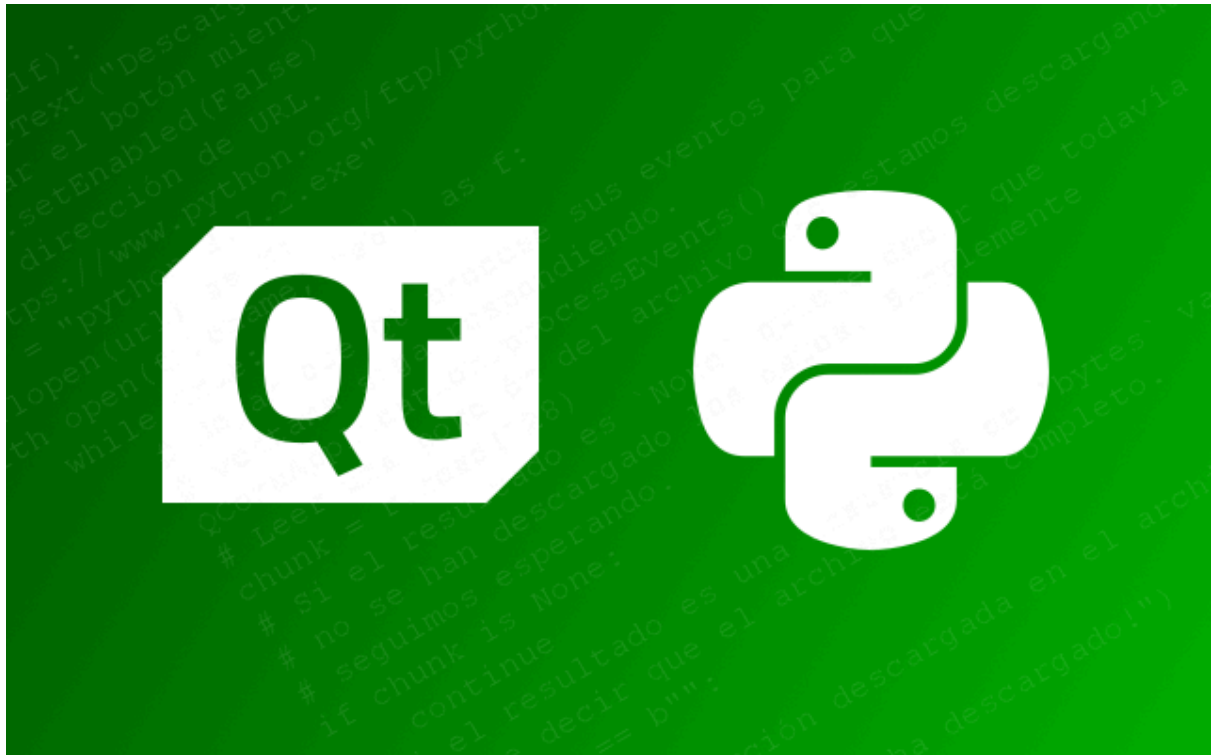


TAREA 2.1 - Explorando nuevos widgets de PySide6

Manuel Parrado Torres



ÍNDICE

1. QRadioButton

- 1 Descripción general
- 2 Principales métodos
- 3 Señales más frecuentes
- 5 Webgrafía

2. QTabWidget

- 1 Descripción general
- 2 Principales métodos
- 3 Señales más frecuentes
- 4 Captura de pantalla
- 5 Webgrafía

3.QProgressBar

- 1 Descripción general
- 2 Principales métodos
- 3 Señales más frecuentes
- 4 Captura de pantalla
- 5 Webgrafía

4. QDateTimeEdit

- 1 Descripción general
- 2 Principales métodos
- 3 Señales más frecuentes
- 4 Captura de pantalla
- 5 Webgrafía

5. QSlider

- 1 Descripción general
- 2 Principales métodos
- 3 Señales más frecuentes
- 4 Captura de pantalla
- 5 Webgrafía

6. QDial

- 1 Descripción general
- 2 Principales métodos
- 3 Señales más frecuentes
- 4 Captura de pantalla
- 5 Webgrafía

1. QRadioButton

1 Descripción general

El QRadioButton es un botón de opción que permite seleccionar solo una alternativa entre varias. Normalmente se agrupan en conjuntos donde solo un botón puede estar activo a la vez. Se usa mucho en formularios, encuestas o configuraciones con opciones excluyentes (por ejemplo: elegir un idioma o un método de pago).

2 Principales métodos

`.setChecked(bool)`: Marca o desmarca el botón.

`.isChecked()`: Devuelve True si el botón está seleccionado.

`.setText(str)`: Cambia el texto que se muestra junto al botón.

`.text()`: Obtiene el texto actual del botón.

`.setAutoExclusive(bool)`: Controla si el botón forma parte de un grupo exclusivo.

`.setEnabled(bool)`: Habilita o deshabilita el botón.

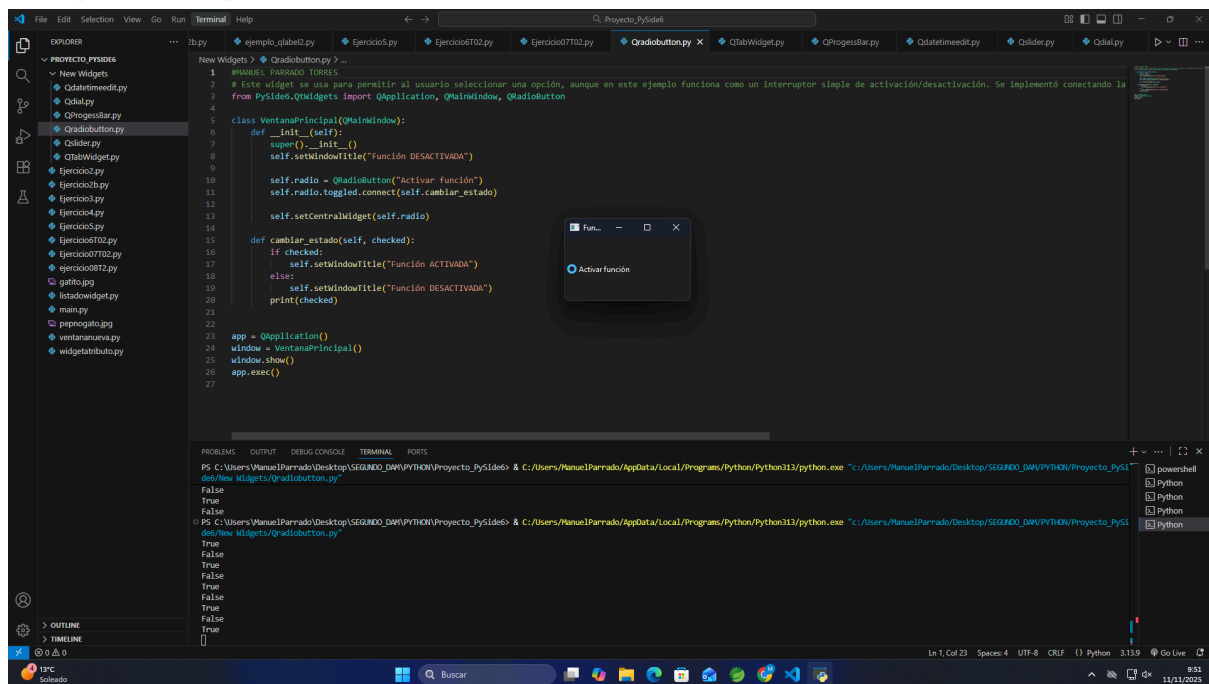
`.setVisible(bool)`: Muestra u oculta el botón.

3 Señales más frecuentes

`toggled(bool)` Se emite cuando el botón cambia de estado (activado/desactivado).

`clicked(bool)` Se emite cuando el usuario hace clic sobre el botón.

4 Captura de pantalla



5 Webgrafía

- <https://doc.qt.io/qtforpython-6/PySide6/QtWidgets/QRadioButton.html>
- <https://www.pythonguis.com/tutorials/pyside6-radio-buttons/>

2. QTabWidget

1 Descripción general

QTabWidget permite organizar el contenido en pestañas (tabs), muy útil cuando una ventana tiene varias secciones relacionadas. Cada pestaña puede contener sus propios controles o layouts, como si fueran mini formularios dentro de la aplicación.

2 Principales métodos

`.addTab(widget, str)`: Añade una nueva pestaña con un widget y un título.

`.insertTab(index, widget, str)`: Inserta una pestaña en una posición concreta.

`.removeTab(index)`: Elimina una pestaña.

`.setTabText(index, str)`: Cambia el título de una pestaña existente.

`.tabText(index)`: Obtiene el texto del título de una pestaña.

`.setCurrentIndex(index)`: Cambia la pestaña activa.

`.currentIndex()`: Devuelve el índice de la pestaña actual.

`.count()`: Devuelve cuántas pestañas hay.

`.widget(index)`: Devuelve el widget contenido en una pestaña.

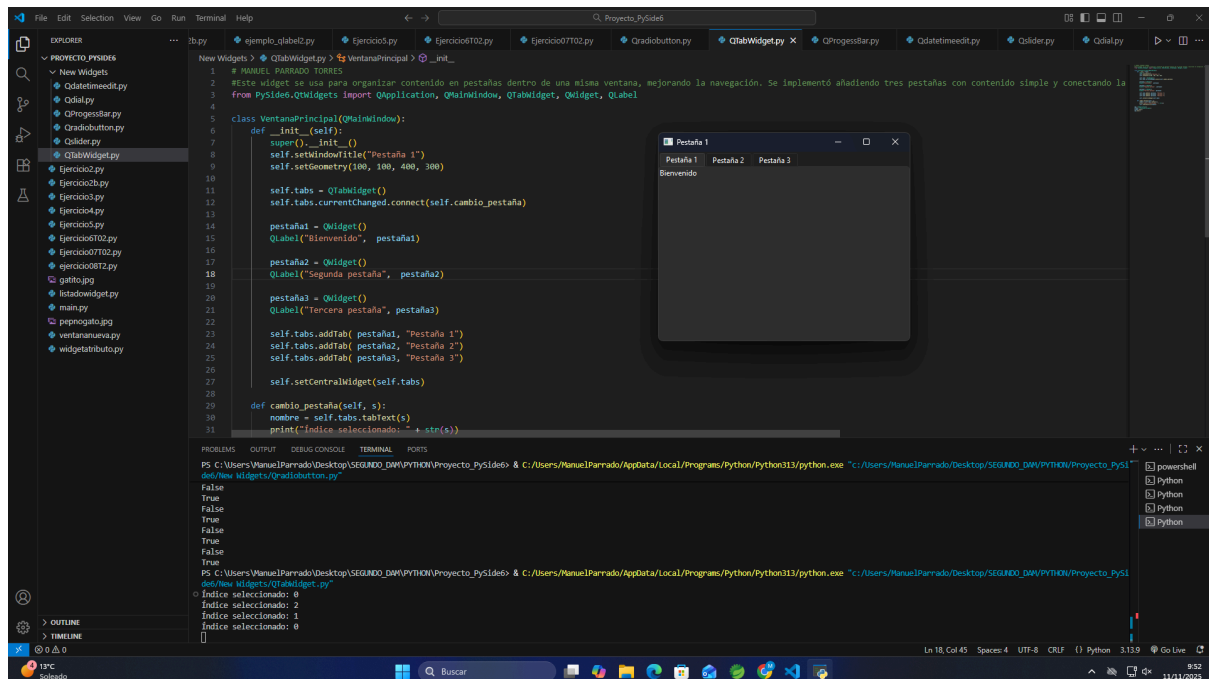
`.setTabsClosable(bool)`: Permite mostrar botones de cierre en las pestañas.

3 Señales más frecuentes

`currentChanged(int)`: Se emite cuando cambia la pestaña activa.

`tabCloseRequested(int)`: Se emite cuando el usuario solicita cerrar una pestaña.

4 Captura de pantalla



5 Webgrafía

- <https://doc.qt.io/qtforpython-6/PySide6/QtWidgets/QTabWidget.html>
- <https://pythonspot.com/en/pyside6-tabs/>

3.QProgressBar

1 Descripción general

QProgressBar muestra visualmente el progreso de una tarea o el porcentaje de avance de una acción. Es común en operaciones que tardan un tiempo, como descargas, cargas o procesos de instalación.

2 Principales métodos

.setValue(int) → Define el valor actual del progreso.

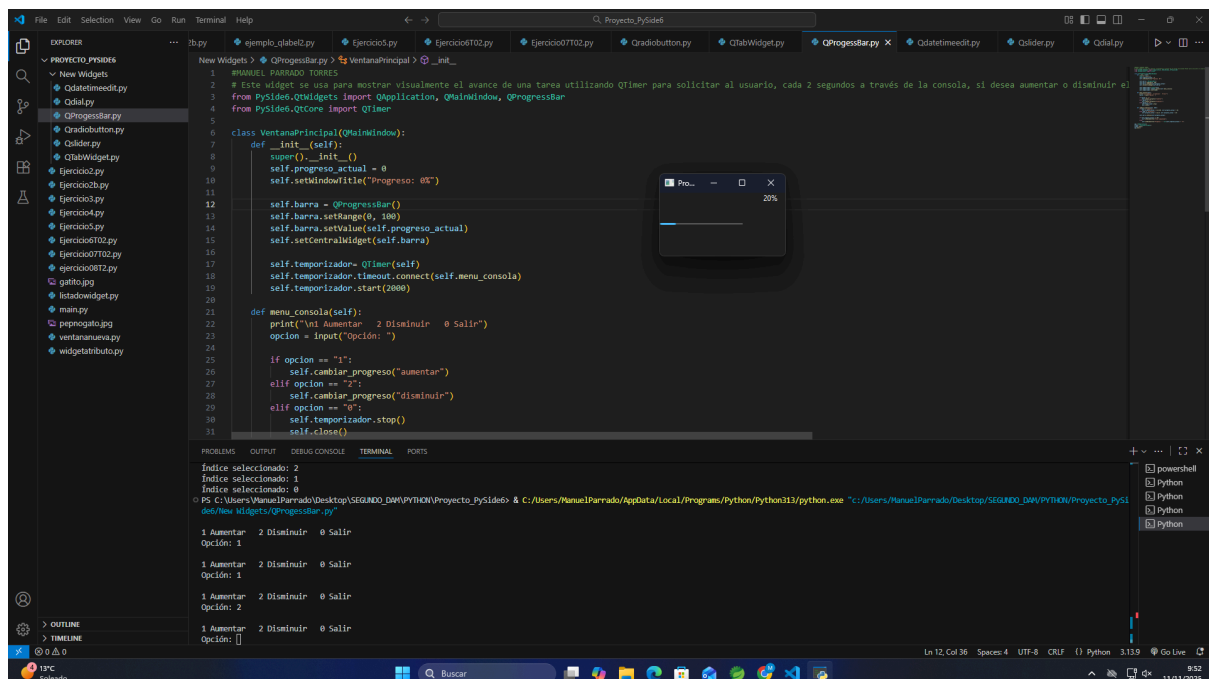
.setRange(min, max) → Fija los límites mínimo y máximo (por ejemplo, 0–100).

.reset() → Vuelve el progreso al estado inicial.

3 Señales más frecuentes

No tiene señales propias, pero normalmente **se actualiza en respuesta a señales de otros widgets**, como un botón o un temporizador.

4 Captura de pantalla



5 Webgrafía

- <https://doc.qt.io/qtforpython-6/PySide6/QtWidgets/QProgressBar.html>
- <https://pythonbasics.org/pyside6-progressbar/>

4. QDateTimeEdit

1 Descripción general

QDateTimeEdit es un widget que permite mostrar y modificar fechas y horas. Se puede usar para seleccionar una fecha de nacimiento, programar una cita o registrar la hora actual. Muestra un campo editable con flechas para ajustar cada parte (día, mes, año, hora, minuto).

2 Principales métodos

`.setDateTime(QDateTime)` → Asigna una fecha y hora inicial.

`.dateTime()` → Devuelve el valor actual del campo.

`.setDisplayFormat(str)` → Cambia el formato mostrado, por ejemplo "dd/MM/yyyy HH:mm".

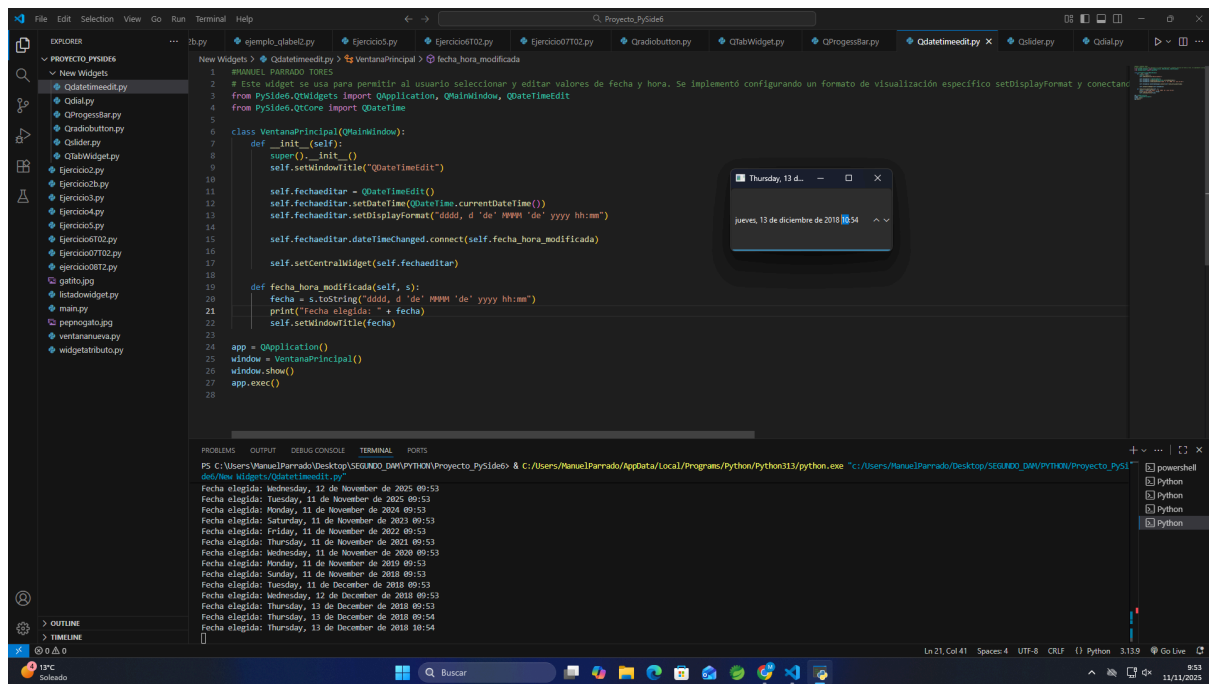
3 Señales más frecuentes

`dateChanged(QDate)` → Se emite cuando cambia la fecha.

`timeChanged(QTime)` → Se emite cuando cambia la hora.

`dateTimeChanged(QDateTime)` → Se emite al cambiar fecha u hora.

4 Captura de pantalla



5 Webgrafía

- <https://doc.qt.io/qtforpython-6/PySide6/QtWidgets/QDateTimeEdit.html>
- <https://pythonspot.com/en/pyside6-date-time/>

5. QSlider

1 Descripción general

QSlider es un control deslizante que permite seleccionar un valor numérico dentro de un rango moviendo una barra horizontal o vertical. Es ideal para ajustar volúmenes, brillo, zoom, etc.

2 Principales métodos

`setMinimum(int)` y `setMaximum(int)` → Definen los límites.

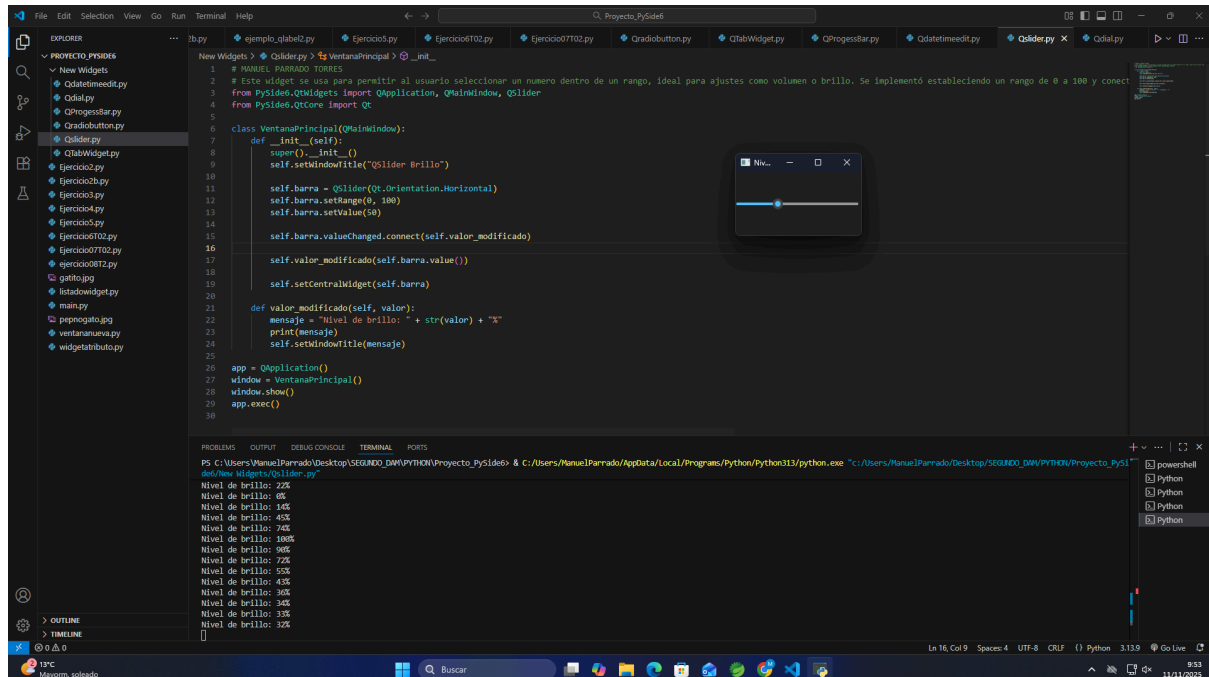
`.setValue(int)` → Asigna un valor actual.

`.value()` → Devuelve el valor actual del control.

3 Señales más frecuentes

valueChanged(int) → Se emite cada vez que el usuario mueve el control.
Se usa para actualizar etiquetas o valores en pantalla.

4 Captura de pantalla



5 Webgrafía

- <https://doc.qt.io/qtforpython-6/PySide6/QtWidgets/QSlider.html>
- <https://pythonbasics.org/pyside6-slider/>

6. QDial

1 Descripción general

QDial es un control giratorio que funciona como un QSlider, pero con forma circular. Se usa en interfaces que imitan controles físicos, como perillas de volumen o ajuste de velocidad.

2 Principales métodos

.setNotchesVisible(True) → Muestra pequeñas marcas alrededor del dial.

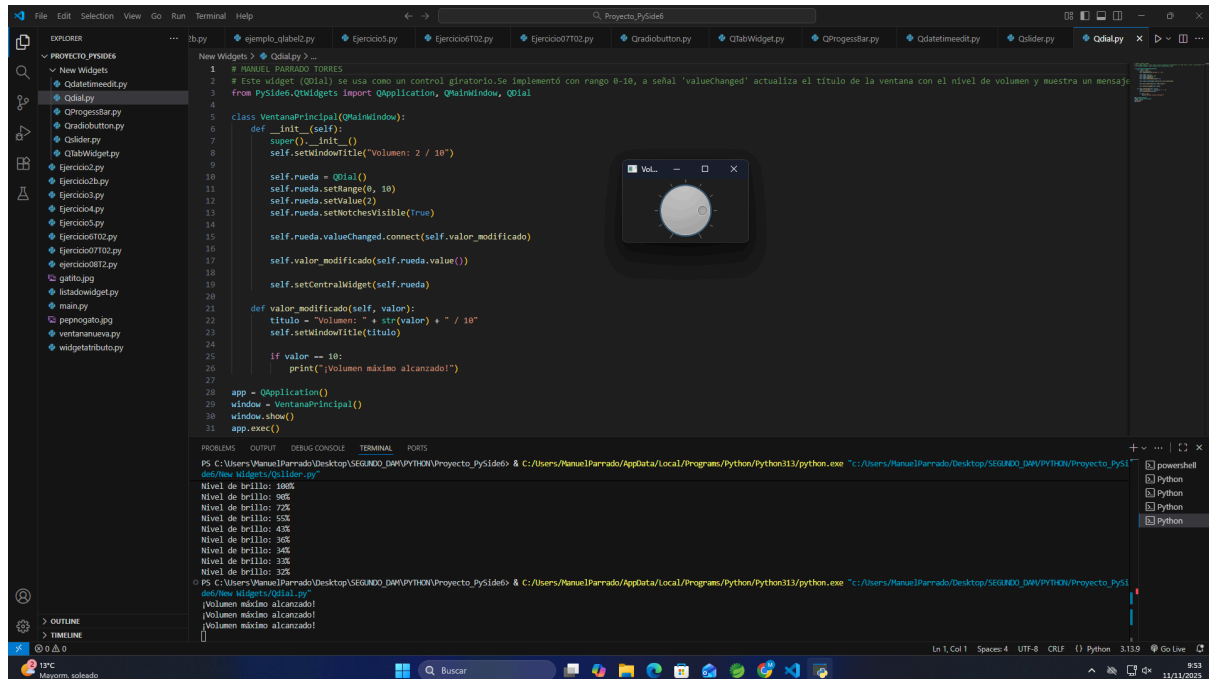
.setRange(min, max) → Define los valores posibles.

.setValue(int) / .value() → Ajusta o lee el valor actual.

3 Señales más frecuentes

valueChanged(int) → Se emite cuando el valor cambia al girar el control.
Ideal para sincronizarlo con etiquetas o barras de progreso.

4 Captura de pantalla



5 Webgrafía

- <https://doc.qt.io/qtforpython-6/PySide6/QtWidgets/QDial.html>
- <https://pythonspot.com/en/pyside6-dial-widget/>